

Das GEEF-Pattern: Policy-gesteuerte Feedback-Loops für KI-Agent-Workflows

Ein Framework für deterministische Qualitätssicherung in der generativen KI

Abstract

Generative KI-Systeme liefern nicht-deterministische Ergebnisse. Ein einzelner LLM-Aufruf garantiert weder Korrektheit noch Konsistenz. Das GEEF-Pattern (Grounding → Execution → Evaluation → Finalize) formalisiert einen typisierten, beobachtbaren Feedback-Loop, der KI-Ausgaben iterativ durch unabhängige Reviewer prüft und nachbessern lässt — gesteuert durch konfigurierbare Konvergenz-Policies. Dieser Artikel stellt die Architektur, die theoretischen Grundlagen und eine Referenzimplementierung in .NET 8 vor.

1. Problemstellung

1.1 Das Qualitätsproblem generativer KI

Large Language Models (LLMs) sind probabilistisch. Selbst bei identischen Prompts und Temperatur 0 sind die Ausgaben nicht deterministisch. In produktiven Anwendungen führt das zu mehreren Problemen:

- **Inkorrekte Ausgaben:** Halluzinationen, factual errors, logische Fehler
- **Inkonsistente Qualität:** Gleicher Prompt, unterschiedliche Ergebnisqualität
- **Schwer validierbare Ergebnisse:** Kein Kompilierfehler, kein Stacktrace — nur menschliches Urteil
- **Fehlende Feedback-Schleife:** Ein einzelner LLM-Aufruf hat keine Möglichkeit zur Selbstkorrektur

1.2 Existierende Ansätze und ihre Grenzen

Ansatz	Stärke	Schwäche
Prompt Engineering	Schnell, keine Infrastruktur	Keine Qualitätsgarantie, fragil
Chain-of-Thought	Bessere Reasoning-Qualität	Kein externer Qualitätscheck
Multi-Agent-Systeme	Arbeitsteilung möglich	Oft unstrukturiert, schwer debugbar
Human-in-the-Loop	Höchste Qualität	Nicht skalierbar, teuer
Einfache Retry-Loops	Einfach zu implementieren	Kein gerichteter Fortschritt, keine Konvergenzgarantie

1.3 Die Lücke

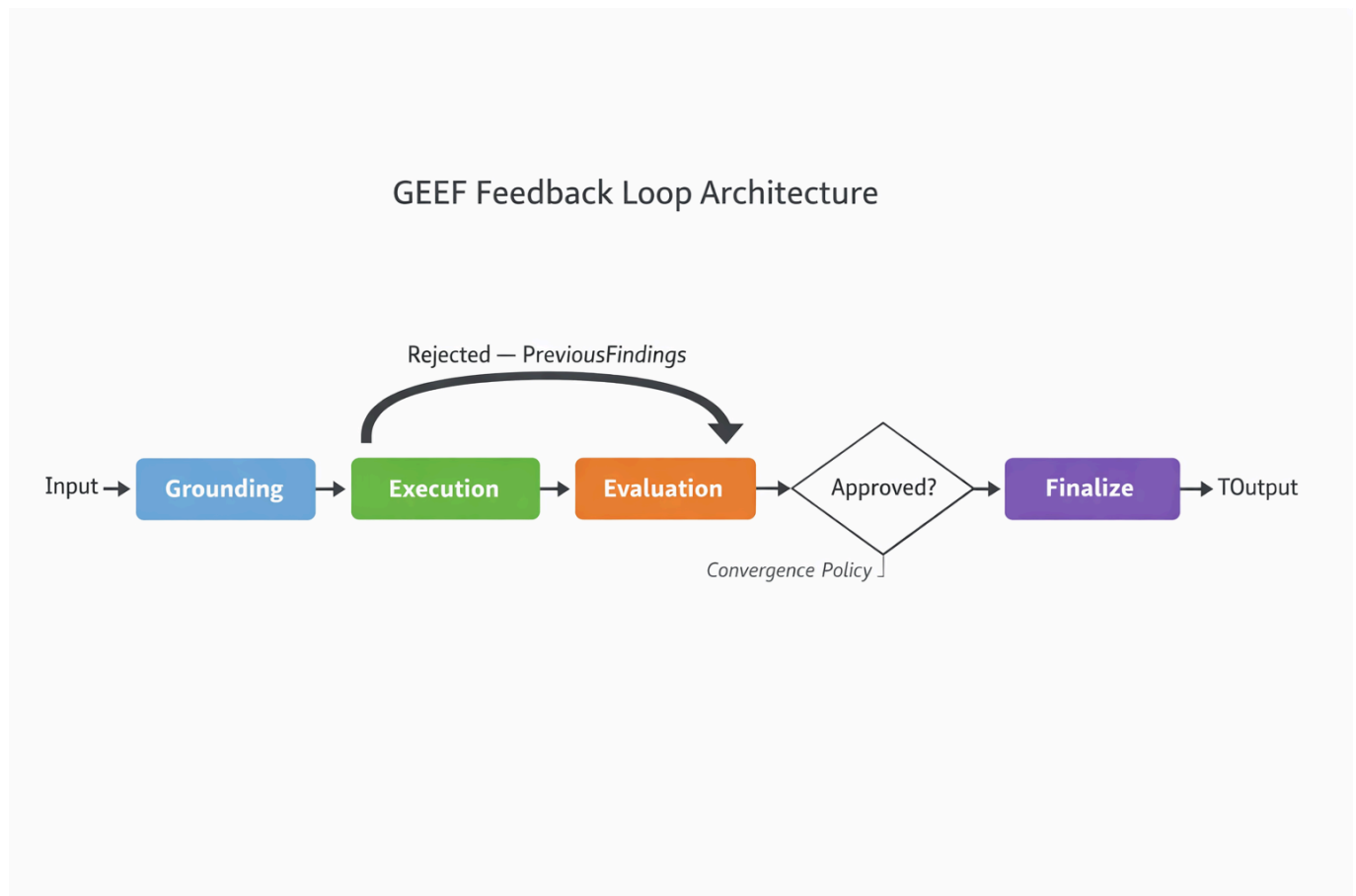
Es fehlt ein formalisiertes Pattern, das:

- KI-Ausgaben **systematisch** durch unabhängige Prüfer validiert
- **Feedback** aus gescheiterten Prüfungen an den Generator zurückgibt
- **Konvergenz** erkennt (Fortschritt, Stagnation, Regression)
- **Observable** ist (Events, Tracing, Logging)
- **Typsicher** und **deterministisch testbar** bleibt

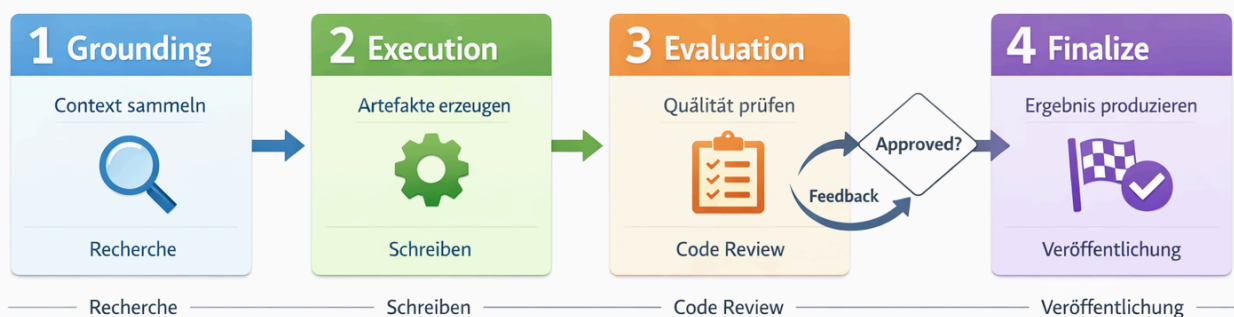
2. Das GEEF-Pattern

2.1 Definition

GEEF ist ein **typisierter, policy-gesteuerter Feedback-Loop** mit vier Phasen:



Phase	Verantwortung	Analogie
Grounding	Kontext sammeln (RAG, Dateien, APIs)	„Recherche“
Execution	Artefakte erzeugen oder überarbeiten	„Schreiben“
Evaluation	Unabhängige Qualitätsprüfung	„Code Review“
Finalize	Typisiertes Endergebnis produzieren	„Veröffentlichung“



2.2 Kernprinzipien

Immutabilität

Der **RunContext** ist unveränderlich. Jede Phase arbeitet mit einem Snapshot und erzeugt einen neuen. Es gibt keine Seiteneffekte zwischen Phasen — das ist die Grundlage für:

- Thread-sichere parallele Reviewer
- Deterministische Reproduzierbarkeit
- Lückenlose Audit-Trails

Separation of Concerns

Jede Phase hat eine klar definierte Aufgabe:

- Der **Executor** weiß nichts über Reviewer
- **Reviewer** wissen nichts voneinander

- Die **Konvergenz-Policy** entscheidet, nicht die Reviewer

Konfigurierbare Konvergenz

Die Entscheidung „Weiter iterieren oder abbrechen?“ wird nicht fest kodiert, sondern durch eine austauschbare **Konvergenz-Policy** getroffen. Das ermöglicht domänenspezifische Strategien.

2.3 Der Feedback-Mechanismus

Der entscheidende Unterschied zu einfachen Retry-Loops: GEEF gibt dem Executor **gerichtetes Feedback**.

Nach einer gescheiterten Evaluation werden die `PreviousFindings` in den Context geschrieben. Der Executor liest diese Findings und kann gezielt nachbessern:

```
Iteration 1:
  Execution → "Die Erde ist 6000 Jahre alt" (halluziniert)
  Reviewer  → Rejected: "Fingerprint: factual-error, Message: Alter der Erde ist ~4.5 Mrd Jahre"

Iteration 2:
  Execution liest PreviousFindings → [factual-error: "Alter der Erde ist ~4.5 Mrd Jahre"]
  Execution → "Die Erde ist ca. 4.5 Milliarden Jahre alt"
  Reviewer  → Approved
```

Das Feedback ist **strukturiert** (nicht nur „nochmal versuchen“) und **fingerprinted** (für Konvergenzanalyse).

3. Konvergenzanalyse

3.1 Finding-Fingerprints

Jeder Finding hat einen eindeutigen **Fingerprint** — typischerweise ein Hash aus Reviewer-Name, Kategorie und normalisierter Nachricht. Die Menge der Fingerprints einer Iteration ermöglicht formale Konvergenzanalyse:

Sei $F(i)$ die Menge der Fingerprints in Iteration i .

Progress	Stagnation	Regression
Iteration 1: A B - C	Iteration 1: A = B = C	Iteration 1: A = B
Iteration 2: A B	Iteration 2: A = B = C	Iteration 2: A ✓
Iteration 3: A	Iteration 3: A = B = C	Iteration 3: A = B - ⚠
 Findings decrease — progress	Same findings repeat — no improvement	Fixed issue reappears

3.2 Fortschritt

Fortschritt liegt vor, wenn $F(i) \neq F(i-1)$ und $|F(i)| < |F(i-1)|$. Die Findings ändern sich und werden weniger.

3.3 Stagnation

Stagnation liegt vor, wenn $F(i) = F(i-1) = \dots = F(i-k+1)$ für einen konfigurierbaren Schwellwert k . Dieselben Findings wiederholen sich — der Executor macht keinen Fortschritt.

Formale Definition:

```
IsStagnant(k)  $\iff \forall j \in [i-k+1, i]: F(j) = F(i)$ 
```

3.4 Regression

Regression liegt vor, wenn ein Fingerprint verschwand und wieder auftaucht:

```
HasRegression()  $\iff \exists fp: fp \in F(i) \wedge fp \notin F(i-1) \wedge fp \in F(i-2)$ 
```

Das deutet auf instabile Fixes hin — der Executor „flickt“ ein Problem, bricht dabei aber etwas anderes.

3.5 Konvergenz-Entscheidungsbaum



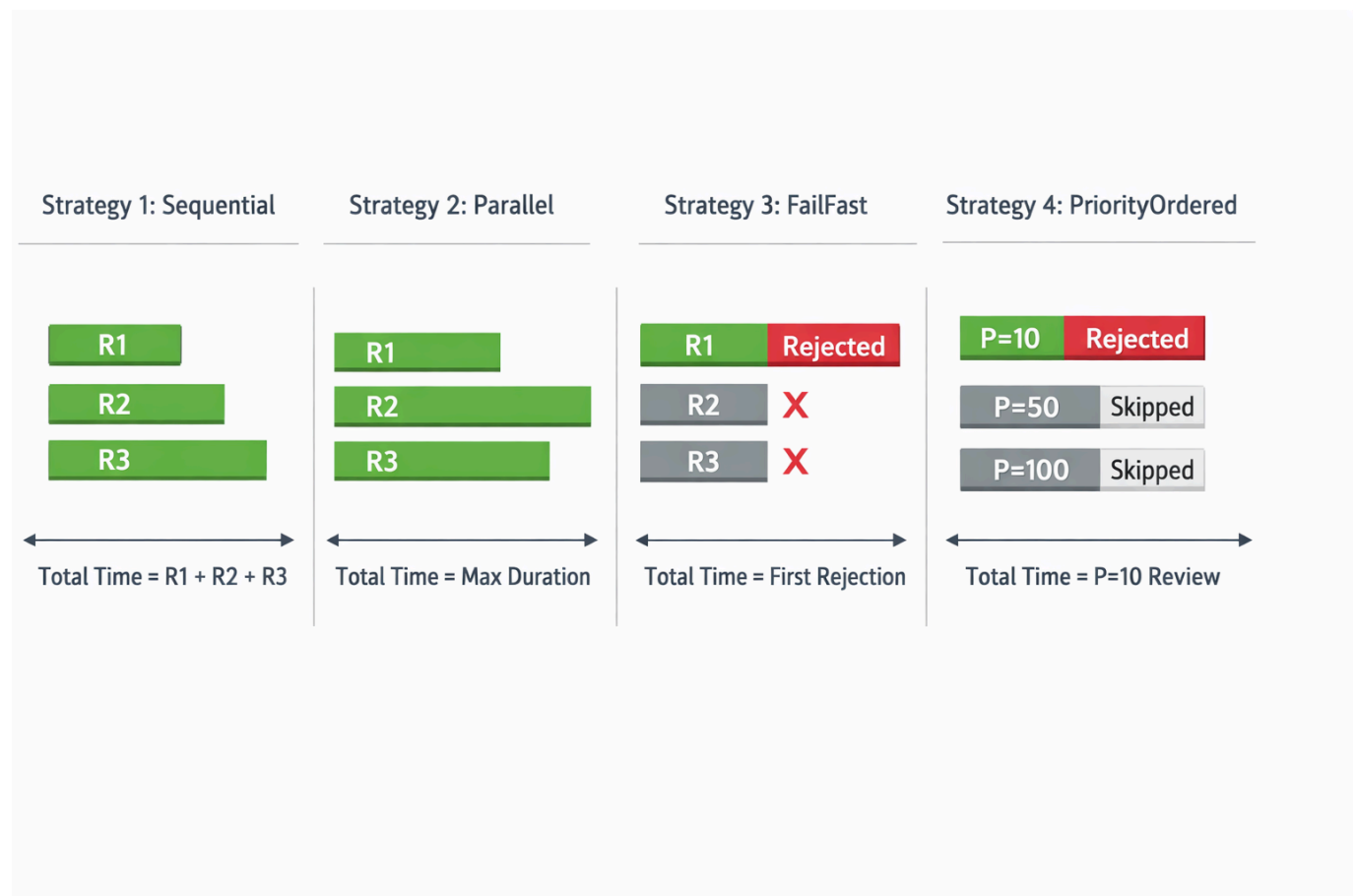
4. Evaluation-Strategien

4.1 Das Reviewer-Problem

Mehrere Reviewer müssen koordiniert werden. Die Fragestellung ist analog zum Task-Scheduling:

- **Latenz:** Wie schnell liegt das Gesamtergebnis vor?
- **Kosten:** Wie viele Reviewer-Aufrufe werden tatsächlich ausgeführt?
- **Fehlertoleranz:** Was passiert bei Reviewer-Ausfällen?

4.2 Strategievergleich



Strategie	Latenz	Kosten	Determinismus	Anwendungsfall
Sequential	Σ Dauer	N Aufrufe	Ja	Debugging, einfache Fälle
Parallel	max Dauer	N Aufrufe	Nein (Reihenfolge)	Schnellste Gesamtzeit
FailFast	$\leq$ max Dauer	$\leq$ N Aufrufe	Nein	Teure KI-Reviewer
PriorityOrdered	$\leq \Sigma$ Dauer	$\leq$ N Aufrufe	Ja	Mix aus günstigen/teuren Checks

4.3 FailFast — Ökonomische Optimierung

FailFast startet alle Reviewer parallel, bricht aber beim ersten blockierenden Ergebnis sofort ab. Die Semantik ist entscheidend:

1. Alle Tasks starten gleichzeitig
2. `Task.WhenAny` -Loop wartet auf die schnellste Fertigstellung
3. Bei `Rejected/Failed`: CancellationToken auslösen, **sofort zurückkehren**
4. Verbleibende Tasks werden non-blocking beobachtet (Exception-Observation)

Das spart Kosten, wenn teure Reviewer (z.B. GPT-4-basierte Code-Reviews) unnötig werden.

4.4 PriorityOrdered — Gestaffelte Prüfung

Idee: Günstige syntaktische Checks (Regex, Compiler-Aufruf) vor teuren semantischen Prüfungen (LLM-Review).

```
Priorität 10: Syntax-Check (ms, kostenlos)
  Priorität 20: Typ-Prüfung (ms, kostenlos)
  Priorität 50: Style-Linter (Sekunden, günstig)
  Priorität 100: KI-Code-Review (Minuten, teuer)
```

Bricht ein Reviewer mit Error-Finding ab, laufen nachfolgende Reviewer nicht.

5. Beobachtbarkeit

5.1 Structured Events

GEEF emittiert 13 typisierte Events über den gesamten Pipeline-Lifecycle. Events sind:

- **Typisiert:** Jeder Event-Typ ist ein eigener Record mit spezifischen Properties
- **Korreliert:** Jeder Event enthält eine RunId
- **Async:** Events werden über `IGeefEventSink` verteilt
- **Erweiterbar:** Composite-Pattern für mehrere Sinks

5.2 Distributed Tracing

Eine OpenTelemetry-kompatible `ActivitySource` erzeugt Spans für jede Phase. Das ermöglicht:

- Flamegraphs über den gesamten Pipeline-Verlauf
- Latenz-Analyse pro Phase und Iteration
- Korrelation mit externen Systemen (HTTP-Aufrufe zu LLM-APIs)

5.3 Three Pillars Integration

Pillar	GEEF-Mechanismus
Logging	<code>LoggingEventSink</code> über <code>ILogger</code>
Tracing	<code>ActivitySource("Geef.Sdk")</code>
Metrics	Custom <code>IGeefEventSink</code> für Prometheus/Grafana

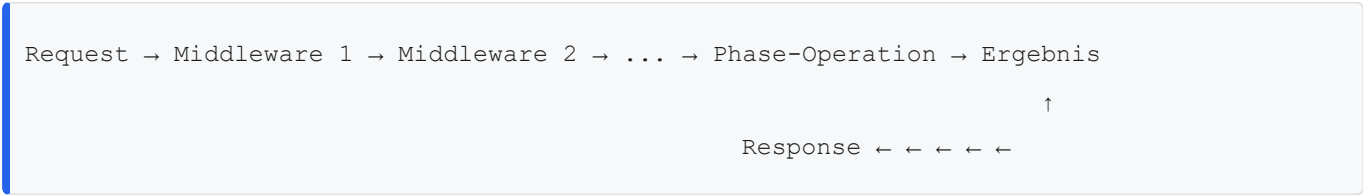
6. Architektur der Referenzimplementierung

6.1 Technologie-Stack

- **.NET 8 / C# 12** — Für Records, Pattern Matching, Primary Constructors
- **ImmutableDictionary** — Persistent Data Structure für den Context
- **System.Diagnostics.ActivitySource** — OpenTelemetry-kompatibles Tracing
- **Microsoft.Extensions.DependencyInjection** — Standard-DI-Integration
- **Keine LLM-SDK-Abhängigkeit** — Bring-your-own-LLM

6.2 Middleware-Pipeline

Inspiriert von ASP.NET Core Middleware:



Middleware kann:

- **Vor** der Phase Code ausführen (Logging, Tracing starten)
- **Nach** der Phase Code ausführen (Metriken, Tracing beenden)
- Die Phase **abbrechen** (Timeout, Circuit Breaker)
- Den Cancellation-Token **ersetzen** (Phase-spezifische Timeouts)

6.3 Builder-Pattern

Fluent API mit Validierung zur Build-Zeit:

```

var pipeline = Geef.CreatePipeline<TOutput>()
    .UseGrounding(...)           // Pflicht
    .UseExecution(...)           // Pflicht
    .AddReviewer(...)            // Mindestens einer
    .UseFinalizer(...)           // Pflicht
    .UseConvergencePolicy(...)   // Optional (Default: 10 Iterationen, 30 min)
    .UseEvaluationStrategy(...)  // Optional (Default: Sequential)
    .UseMiddleware(...)          // Optional
    .ConfigureEvents(...)        // Optional
    .Build();                     // Validiert, wirft PipelineConfigurationException

```

6.4 Exception-Design

Fünf Exceptions bilden eine klare Hierarchie:

Exception	Wann	Wiederherstellbar?
<code>PipelineConfigurationException</code>	Build-Zeit: fehlende Komponente	Nein (Programmierfehler)
<code>ProviderException</code>	Laufzeit: Infrastrukturfehler	Möglicherweise (Retry)
<code>PhaseTimeoutException</code>	Laufzeit: Timeout einer Phase	Ja (längeres Timeout)
<code>ConvergenceFailedException</code>	Laufzeit: Terminale Konvergenz-Entscheidung (MaxAttempts, Stagnation, Regression, CriticalBlocker, Eskalation)	Ja (mehr Iterationen, besserer Prompt)
<code>OperationCanceledException</code>	Externes Cancellation	Ja (erneuter Aufruf)

7. Anwendungsfälle

7.1 KI-gestützte Code-Generierung

```
Grounding: Repository-Kontext laden, relevante Dateien finden
Execution: LLM generiert Code basierend auf Spec + Kontext + Findings
Reviewer 1: Syntax-Check (Kompilierung)
Reviewer 2: Statische Analyse (Roslyn Analyzers)
Reviewer 3: KI-Code-Review (Security, Best Practices)
Reviewer 4: Testabdeckung prüfen
Finalize: Code formatieren, Dateien schreiben
```

7.2 Dokumentations-Generierung

```
Grounding: Quellcode und bestehende Docs laden
Execution: LLM generiert/aktualisiert Dokumentation
Reviewer 1: Markdown-Validierung
Reviewer 2: Link-Checker
Reviewer 3: Factual-Accuracy-Check gegen Quellcode
Finalize: Markdown-Dateien schreiben
```

7.3 Datenextraktion und -transformation

```
Grounding: Quelldaten laden, Schema definieren
Execution: LLM extrahiert strukturierte Daten
Reviewer 1: Schema-Validierung (JSON Schema)
Reviewer 2: Plausibilitätsprüfung (Wertebereichs-Checks)
Reviewer 3: Konsistenz-Check gegen bekannte Referenzdaten
Finalize: Daten in Zielformat konvertieren
```

7.4 Multi-Agenten-Orchestrierung

GEEF-Pipelines können verschachtelt werden:

```
// Äußere Pipeline: Gesamt-Orchestrierung
var outer = Geef.CreatePipeline<ProjectResult>()
    .UseExecution(new CompositeExecution(
        innerCodePipeline,      // Innere Pipeline für Code
        innerTestPipeline,      // Innere Pipeline für Tests
        innerDocPipeline))      // Innere Pipeline für Docs
// ...
    .Build();
```

8. Vergleich mit existierenden Ansätzen

8.1 GEEF vs. LangChain/LangGraph

Aspekt	LangChain/LangGraph	GEEF
Fokus	Allgemeine LLM-Chains	Qualitätssichernde Feedback-Loops
Typisierung	Schwach (Python)	Stark (C# Generics)
Konvergenz	Nicht eingebaut	Kern-Feature (Stagnation, Regression)
Immutabilität	Nicht erzwungen	Architektur-Prinzip
Testbarkeit	Framework-abhängig	LLM-unabhängig, delegate-basierte Tests
Observability	Plugin-basiert	Eingebaut (Events, Tracing)

8.2 GEEF vs. AutoGen/CrewAI

Aspekt	AutoGen/CrewAI	GEEF
Paradigma	Multi-Agent-Konversation	Policy-gesteuerter Loop
Steuerung	Emergent (Agenten entscheiden)	Deterministisch (Policy entscheidet)
Debugging	Schwierig (chaotische Konversation)	Einfach (linearer Loop, Events)
Kosten	Unkontrolliert (Agenten reden beliebig)	Kontrolliert (MaxIterations, FailFast)

8.3 GEEF vs. Einfache Retry-Loops

Aspekt	Retry-Loop	GEEF
Feedback	„Versuche es nochmal“	Strukturierte Findings mit Fingerprints
Konvergenz	Max-Retries	Stagnation, Regression, Policies
Prüfung	Eine Prüfung	Mehrere unabhängige Reviewer
Strategie	Keine	Sequential, Parallel, FailFast, Priority
Beobachtbarkeit	printf	Events, Tracing, Middleware

9. Limitierungen und Ausblick

9.1 Aktuelle Limitierungen

- **Linearer Loop:** GEEF unterstützt keine verzweigten Workflows (DAGs). Jede Iteration durchläuft alle Phasen.
- **Stateless zwischen Runs:** Kein eingebautes Langzeitgedächtnis über Pipeline-Läufe hinweg.
- **Einzelne Execution:** Pro Iteration gibt es genau einen Executor. Parallele Generierung wird noch nicht unterstützt.

9.2 Zukünftige Erweiterungen

- **Adaptive Policies:** Konvergenz-Policies, die basierend auf historischen Runs lernen
- **Streaming-Support:** Zwischenergebnisse während der Execution streamen
- **Cost Tracking:** Integriertes Token-/Kosten-Tracking über Iterationen
- **DAG-Support:** Verzweigte Pipelines mit bedingten Pfaden
- **Persistent Memory:** Cross-Run-Gedächtnis für langlebige Aufgaben

10. Fazit

Das GEEF-Pattern adressiert eine fundamentale Herausforderung bei der produktiven Nutzung generativer KI: **die systematische Qualitätssicherung nicht-deterministischer Ausgaben.** Durch die Kombination aus:

- **Immutablem, typisiertem Context** für Reproduzierbarkeit
- **Unabhängigen Reviewern** für Separation of Concerns
- **Konvergenzanalyse** für kontrollierte Loop-Terminierung
- **Austauschbaren Strategien** für domänenspezifische Optimierung
- **Eingebauter Beobachtbarkeit** für Debugging und Monitoring

entsteht ein Framework, das KI-Workflows von „hoffentlich funktioniert es“ zu „nachweisbar geprüft“ hebt.

Die Referenzimplementierung in .NET 8 demonstriert, dass dieses Pattern mit minimalem Overhead und maximaler Typsicherheit umsetzbar ist — ohne Abhängigkeit auf ein spezifisches LLM-SDK.

Literatur und Referenzen

1. Wei, J. et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. NeurIPS.
 2. Yao, S. et al. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR.
 3. Shinn, N. et al. (2023). *Reflexion: Language Agents with Verbal Reinforcement Learning*. NeurIPS.
 4. Madaan, A. et al. (2023). *Self-Refine: Iterative Refinement with Self-Feedback*. NeurIPS.
 5. Microsoft Research. (2023). *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. ArXiv.
 6. OpenTelemetry Specification. (2024). *Tracing API*. <https://opentelemetry.io/docs/specs/otel/trace/>
 7. Fowler, M. (2004). *Inversion of Control Containers and the Dependency Injection pattern*. martinfowler.com.
 8. Okasaki, C. (1998). *Purely Functional Data Structures*. Cambridge University Press.
-

Geef.Sdk v1.0.0 — Open Source, .NET 8, C# 12 Repository: github.com/chromcom/geef